

# Relatório Técnico de Engenharia de Prompt

## Experiência 04: Engenharia Reversa Assistida por Inteligência Artificial

|        |                         |             |                            |
|--------|-------------------------|-------------|----------------------------|
| Aluno: | Adriano Salviano Marçal | Disciplina: | Engenharia de Prompt em IA |
| Curso: | Ciência da Computação   | Data:       | 16 de Maio de 2026         |

### 1. Introdução e Contextualização

O avanço vertiginoso dos Modelos de Linguagem de Grande Porte (LLMs) remodelou o ecossistema de desenvolvimento de software. A disciplina de Engenharia de Prompt em IA estuda não apenas a formulação de instruções otimizadas para obtenção de respostas textuais, mas também a aplicação dessas ferramentas na automação, arquitetura e aceleração do ciclo de vida de sistemas computacionais.

Este relatório técnico documenta os resultados e as reflexões da **Experiência 04: Engenharia Reversa**, cujo propósito central consistiu em atuar como um Desenvolvedor de Software Assistido por IA. O desafio prático envolveu a reconstrução de um aplicativo funcional baseado estritamente na observação de sua interface externa e comportamento lógico, sem qualquer acesso direto ao seu código-fonte original ou fornecimento de hiperlinks estruturais para o modelo generativo.

### 2. Metodologia de Engenharia Reversa Assistida por IA

O procedimento prático seguiu um pipeline rigoroso de três etapas interdependentes, executadas em conjunto com a plataforma Google AI Studio utilizando o modelo Gemini:

- Análise e Mapeamento Funcional:** Exploração minuciosa do ecossistema visual da aplicação de referência (no caso prático, um Editor Markdown Live baseado no *StackEdit*). Mapeamento manual de todos os componentes visuais, interações de usuário e regras de lógica de negócio implícitas na interface.
- Configuração de Diretrizes Sistêmicas (System Instructions):** Definição rigorosa do papel do modelo no Google AI Studio. O LLM foi instruído a atuar estritamente como um Engenheiro Full-Stack Sênior focado na geração de Aplicações de Página Única (SPAs) coesas, robustas e limpas.
- Construção, Execução e Ajuste Iterativo:** Geração do código-fonte completo (HTML5, CSS3 e JavaScript) em arquivo único. O código foi testado em ambiente local, comparado esteticamente com a

aplicação de referência e refinado por meio de prompts incrementais até atingir a paridade funcional e estética.

### 3. Análise Crítica e Fundamentação Teórica

A transição do esforço de desenvolvimento tradicional (focado na escrita puramente sintática de código) para a descrição lógica e conceitual imposta pela IA Generativa traz à tona debates profundos acerca da evolução da carreira técnica e dos limites éticos da tecnologia.

#### 3.1. O Impacto na Formação do Engenheiro de Software Júnior

A automação da escrita de código sintático reduz drasticamente a barreira de entrada técnica e mitiga a necessidade de memorização de APIs e sintaxes complexas. O engenheiro júnior deixa de ser um mero tradutor de requisitos em linhas de código e passa a atuar como um orquestrador de soluções computacionais.

Para evitar a obsolescência profissional frente a ferramentas avançadas como o Gemini, duas competências tornam-se mandatórias na formação acadêmica e profissional em Ciência da Computação:

- **Visão Sistêmica e Arquitetura de Software:** Domínio sobre como os componentes de software se integram, fluxos de dados, padrões de projeto (Design Patterns) e capacidade de traduzir um problema abstrato em uma estrutura modular que a IA consiga processar eficientemente.
- **Revisão Crítica de Código (Code Review) e Depuração Avançada:** Modelos generativos estão sujeitos a alucinações lógicas, vulnerabilidades de segurança e geração de código legado ou ineficiente. O profissional deve possuir fundamentação teórica sólida para auditar, testar, identificar gargalos de performance e corrigir as falhas presentes nas entregas automatizadas.

3.2. O Limiar entre Prototipagem e Plágio Digital

A facilidade de replicação de interfaces completas gera discussões éticas sobre originalidade e propriedade intelectual. A linha que separa a engenharia reversa para aprendizado ou prototipagem rápida da prática de plágio digital torna-se nítida a partir do momento em que o artefato replicado é distribuído comercial ou publicamente sem modificações substanciais de valor.

O plágio digital caracteriza-se pela reprodução fidedigna do *trade dress* (identidade visual, paleta de cores, microinterações) combinada à lógica central de negócios, apropriando-se do esforço de design e experiência do usuário (UX) desenvolvido por outrem para competir de forma desleal.

Como solução ética e diretriz corporativa/acadêmica para proteger a inovação sem frear o avanço das ferramentas de IA, propõe-se a **Diretriz de Transformação Funcional e Visual Obrigatória**:

*"Qualquer software desenvolvido por meio de engenharia reversa assistida por IA deve conter, obrigatoriamente, alterações drásticas em sua camada de interface (UI) e a adição de, no mínimo, uma funcionalidade inovadora inexistente no modelo original. Adicionalmente, o código deve documentar de forma transparente as referências que inspiraram sua criação através de metadados legíveis."*

4. Estudo de Caso Prático: Desenvolvimento do Markdown Live Editor

Como validação prática dos conceitos de Engenharia de Prompt, foi estruturada a especificação e geração de um editor de Markdown em tempo real. A tabela abaixo resume o mapeamento de componentes executado antes da instrução do modelo:

| Componente Visual            | Lógica de Negócio Operacional                                         | Implementação Técnica Proposta                                                           |
|------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Painel de Entrada (Esquerda) | Captura do texto bruto digitado pelo usuário em tempo real.           | Elemento HTML <code>&lt;textarea&gt;</code> com escuta ao evento <code>input</code> .    |
| Painel de Preview (Direita)  | Exibição imediata da renderização do código HTML convertido.          | Elemento <code>&lt;div&gt;</code> container atualizado via <code>innerHTML</code> .      |
| Motor de Parse               | Conversão sintática dos marcadores Markdown para tags HTML legítimas. | Integração da biblioteca externa <code>marked.js</code> via CDN no cabeçalho do arquivo. |

O prompt estruturado abaixo demonstra a aplicação prática das técnicas de contextualização de papel, restrição de escopo e clareza de saída para a geração do artefato sem erros:

```
Instrução Sistemica: Atue como Engenheiro Full-Stack Sênior. Forneça o código em um arquivo HTML único.
```

```
Prompt: Crie um editor "Markdown Live". Interface ocupando 100% da viewport, dividida verticalmente em duas colunas de 50%. A coluna esquerda possui um textarea para escrita em Markdown. A coluna direita é uma div que renderiza o preview em tempo real. Use a biblioteca 'marked.js' via CDN para realizar o parse através de um event listener de input. O design deve ser moderno, clean, com fundo cinza claro (#f4f4f9) e tipografia sans-serif legível.
```

## 5. Conclusão

A Engenharia de Prompt aplicada à Engenharia Reversa comprova que as habilidades mais críticas do cientista da computação moderno migraram da execução operacional para a capacidade analítica e de design abstrato. O sucesso no desenvolvimento assistido por inteligência artificial reside na precisão conceitual do engenheiro, que atua como o validador ético e técnico, garantindo a qualidade, a segurança e a originalidade do software produzido.